

UNIVERSIDAD NACIONAL DE INGENIERÍA



PROFESOR: SELIS VASQUEZ, LUIS ANTONIO

ALUMNOS:

ALARCÓN SOLIS, JEANPIER CONTI (20243026J)

HUAMAN YUPANQUI, JHON KEVIN (20241442F)

CURSO: ALGORITMOS Y ESTRUCTURA DE DATOS (BMA09)

SECCIÓN: T



2026

ÍNDICE

INTRODUCCIÓN.....	4
OBJETIVOS.....	4
ANÁLISIS DE PROBLEMA.....	5
Concepto del Backtracking.....	5
Espacio de búsqueda.....	5
Construcción de Soluciones.....	5
Poda.....	5
Fuerza bruta.....	5
COMPLEJIDAD COMPUTACIONAL.....	6
COMPLEJIDAD TEMPORAL.....	6
COMPLEJIDAD ESPACIAL.....	6
COMPLEJIDAD DE LOS ALGORITMOS DE BACKTRACKING.....	6
APLICACIONES Y COMPARACIÓN.....	6
APLICACIONES DEL BACKTRACKING.....	6
VENTAJAS DEL BACKTRACKING.....	6
DESVENTAJAS Y LIMITACIONES.....	6
COMPARACIÓN CON FUERZA BRUTA.....	6
SITUACIONES DE APLICACIÓN.....	6
ALGORITMOS Y APLICACIONES.....	7
PERMUTACIONES.....	7
DESCRIPCIÓN DEL PROBLEMA.....	7
ESTRATEGIA DE BACKTRACKING.....	7
EJEMPLO Y RESULTADO DE EJECUCIÓN.....	7
COMPLEJIDAD TEMPORAL Y ESPACIAL.....	9
COMBINACIONES.....	9
DESCRIPCIÓN DEL PROBLEMA.....	9
ESTRATEGIA DE BACKTRACKING.....	9
EJEMPLO Y RESULTADO DE EJECUCIÓN.....	9
COMPLEJIDAD TEMPORAL Y ESPACIAL.....	9
N-REINAS.....	9
DESCRIPCIÓN DEL PROBLEMA.....	9
ESTRATEGIA DE BACKTRACKING.....	9
EJEMPLO Y RESULTADO DE EJECUCIÓN.....	9
COMPLEJIDAD TEMPORAL Y ESPACIAL.....	9
SUDOKU.....	9
DESCRIPCIÓN DEL PROBLEMA.....	9
ESTRATEGIA DE BACKTRACKING.....	10
EJEMPLO Y RESULTADO DE EJECUCIÓN.....	10
COMPLEJIDAD TEMPORAL Y ESPACIAL.....	10
COMPARACIÓN DE LOS ALGORITMOS IMPLEMENTADOS.....	11
COMPARACIÓN DE COMPLEJIDAD.....	11
COMPARACIÓN DE CARACTERÍSTICAS.....	11
ANÁLISIS DE LOS RESULTADOS OBTENIDOS.....	11

CONCLUSIONES.....	11
--------------------------	-----------

INTRODUCCIÓN

En el presente informe desarrollaremos la técnica del backtracking para la resolución de problemas utilizando el lenguaje C. Se abarcará el concepto de backtracking, sus procesos, su complejidad, sus aplicaciones en diferentes problemas, además de realizar una comparación de los resultados obtenidos.

OBJETIVOS

Explicar qué estrategias son las más utilizadas así como sus aplicaciones y la comparación entre ellas sobre el tiempo empleado en el desarrollo de los algoritmos y su eficiencia para encontrar una solución.

ANÁLISIS DE PROBLEMA

Concepto del Backtracking

El backtracking es una técnica de resolución de problemas que utiliza enfoques diversos para encontrar la solución. Esta técnica implica intentar soluciones pasos a pasos. Si uno de los intentos nos lleva a una solución inválida, se modifica esa solución y se vuelve a intentarlo. Es una especie de análisis por ensayo y error.

Espacio de búsqueda

El espacio de búsqueda define todas las posibles soluciones o estados que deben tenerse en cuenta para resolver un problema concreto utilizando la técnica de backtracking. Es una forma de representar estas soluciones a través de un árbol de búsqueda que contiene todos los pasos que deben darse para resolver el problema.

Construcción de Soluciones

El proceso de encontrar soluciones mediante esta técnica se realiza gradualmente, paso a paso. Cada nivel del árbol añade nuevos elementos a las soluciones ya encontradas. En otras palabras, en cada nivel se toma una opción concreta y se avanza hasta que se da con una solución incorrecta o hasta que se encuentre la solución correcta. Si llega a ser incorrecta, se vuelve atrás y se sigue buscando.

Poda

Es un mecanismo del algoritmo que consiste en descartar ramas completas del árbol de búsqueda de soluciones al detectar errores futuros que no permiten una solución válida, por lo tanto nos evita desarrollar miles de posibilidades dentro de esas ramas y ahorrarnos el tiempo de búsqueda de la solución.

Fuerza bruta

La fuerza bruta es un método directo para resolver un problema que consiste en probar todas las combinaciones o soluciones posibles hasta encontrar una que cumpla con las condiciones establecidas. Su principal ventaja es que resulta más sencilla de implementar y, en muchos casos, permite encontrar una solución sin utilizar estrategias complejas. Una desventaja es el tiempo en que se toma para encontrar la solución correcta si el número de combinaciones es grande.

COMPLEJIDAD COMPUTACIONAL

Es la rama de informática que nos permite clasificar y analizar los problemas según la cantidad de recursos computacionales que se requiere para resolver un algoritmo en función al tamaño de la entrada.

COMPLEJIDAD TEMPORAL

Mide cuánto crece el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de la entrada n . No se mide en segundos y se expresa con las siguientes notación asintótica

$O(f(n))$: cota superior, el peor caso

$\theta(f(n))$: cota intermedia, coinciden cota superior e inferior

$\Omega(f(n))$: cota inferior, el mejor caso

COMPLEJIDAD ESPACIAL

Mide cuánta memoria adicional necesita un algoritmo para ejecutarse completamente en función de su entrada n . Está compuesta por la memoria de entrada, para los datos de entrada y la memoria auxiliar, para las variables y pilas de llamadas en algoritmos recursivos.

COMPLEJIDAD DE LOS ALGORITMOS DE BACKTRACKING

La técnica del backtracking está desarrollada de forma recursiva, y por tanto su complejidad temporal y espacial tienen parámetros.

Complejidad temporal (backtracking):

Tomamos el peor de los casos, y usando el backtracking exploramos todo el árbol de búsqueda por lo que la su complejidad se convierte en exponencial. Ante esta situación, la poda nos permite reducir notoriamente las cantidad de posibilidades calculadas y mejorar la eficiencia.

Complejidad Espacial (backtracking):

Depende principalmente de la profundidad de la recursión mas no del tamaño del árbol ya que sólo mantenemos la memoria del camino actual gracias a las pilas de recursividad y evitamos almacenar todas las soluciones.

APLICACIONES Y COMPARACIÓN

APLICACIONES DEL BACKTRACKING

Se utiliza en problemas como en la generación de permutaciones, combinaciones, N-Reinas, sudoku, entre otros.

VENTAJAS DEL BACKTRACKING

Permite descartar caminos inválidos anticipadamente.

Reduce la cantidad de posibilidades exploradas respecto a una búsqueda completamente exhaustiva.

Se adapta bien a problemas que pueden representarse como árboles de decisiones(Poda).

Puede encontrar una solución sin necesidad de generar todas las soluciones posibles.

DESVENTAJAS Y LIMITACIONES

Puede tener una complejidad temporal muy elevada.

En problemas grandes puede explorar una cantidad enorme de posibilidades.

Su eficiencia depende bastante de la poda.

Las implementaciones recursivas pueden consumir memoria por la pila de llamadas.

COMPARACIÓN CON FUERZA BRUTA

Si bien la fuerza bruta te llega a mostrar el resultado, el procedimiento es tedioso a diferencia de el Backtracking que en su proceso de solución cuando un camino no es viable no regresa al inicio sino que retrocede un paso y toma otro camino así evitando hacer el proceso nuevamente y disminuyendo el tiempo en que se demora.

SITUACIONES DE APLICACIÓN

El Backtracking resulta apropiado cuando un problema puede dividirse en una secuencia de decisiones y es posible determinar durante la construcción de una solución que una determinada elección no puede conducir a una solución válida.

ALGORITMOS Y APLICACIONES

PERMUTACIONES

DESCRIPCIÓN DEL PROBLEMA

Se tiene un conjunto de (n) elementos, por lo que se desea generar todas las posibles ordenaciones de dichos elementos.

Ejemplo:

Para el conjunto:

$$\{1, 2, 3\}$$

Las posibles permutaciones son:

$$123, 132, 213, 231, 312, 321$$

Como se utilizan todos los elementos sin repetir, el número total de permutaciones es:

$$n!$$

Por lo tanto, para $(n=3)$:

$$3! = 3 \times 2 \times 1 = 6$$

En consecuencia, existen 6 posibles ordenaciones.

ESTRATEGIA DE BACKTRACKING

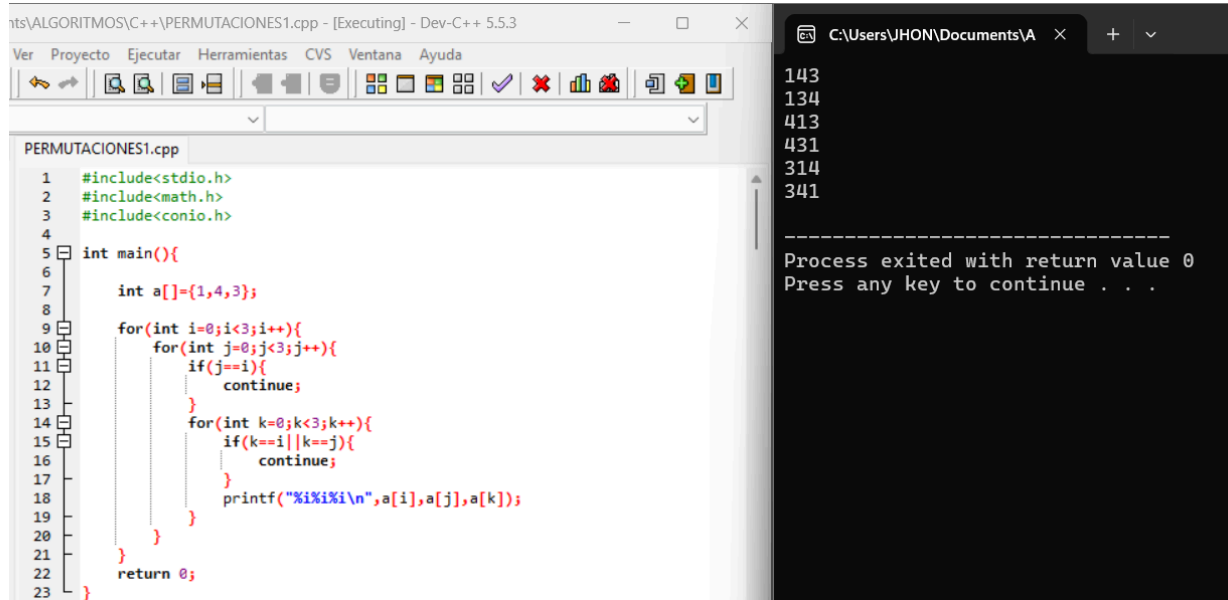
Se implementa un algoritmo utilizando la técnica de Backtracking. El algoritmo selecciona un elemento para la posición actual y continúa construyendo la permutación de manera recursiva. Cuando se alcanza una solución completa, esta se registra. Luego, el algoritmo

“Año de la Esperanza y el Fortalecimiento de la Democracia”

retrocede y deshace lo que se realizó, permitiendo probar otra alternativa y explorar las demás posibilidades.

EJEMPLO Y RESULTADO DE EJECUCIÓN

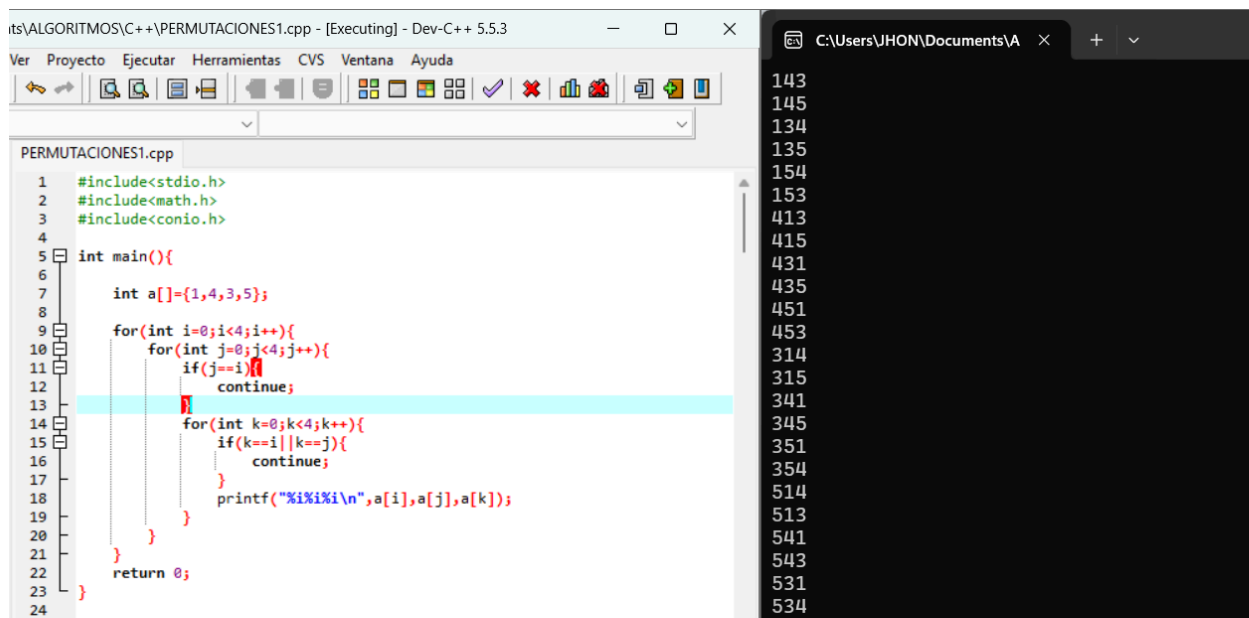
Se implementaron 2 algoritmos para comparar la generación de permutaciones sin utilizar Backtracking y utilizando Backtracking.



```
1 #include<stdio.h>
2 #include<math.h>
3 #include<conio.h>
4
5 int main(){
6     int a[]={1,4,3};
7
8     for(int i=0;i<3;i++){
9         for(int j=0;j<3;j++){
10             if(j==i){
11                 continue;
12             }
13             for(int k=0;k<3;k++){
14                 if(k==i || k==j){
15                     continue;
16                 }
17                 printf("%i%i%i\n",a[i],a[j],a[k]);
18             }
19         }
20     }
21     return 0;
22 }
```

```
143
134
413
431
314
341

-----
Process exited with return value 0
Press any key to continue . . .
```

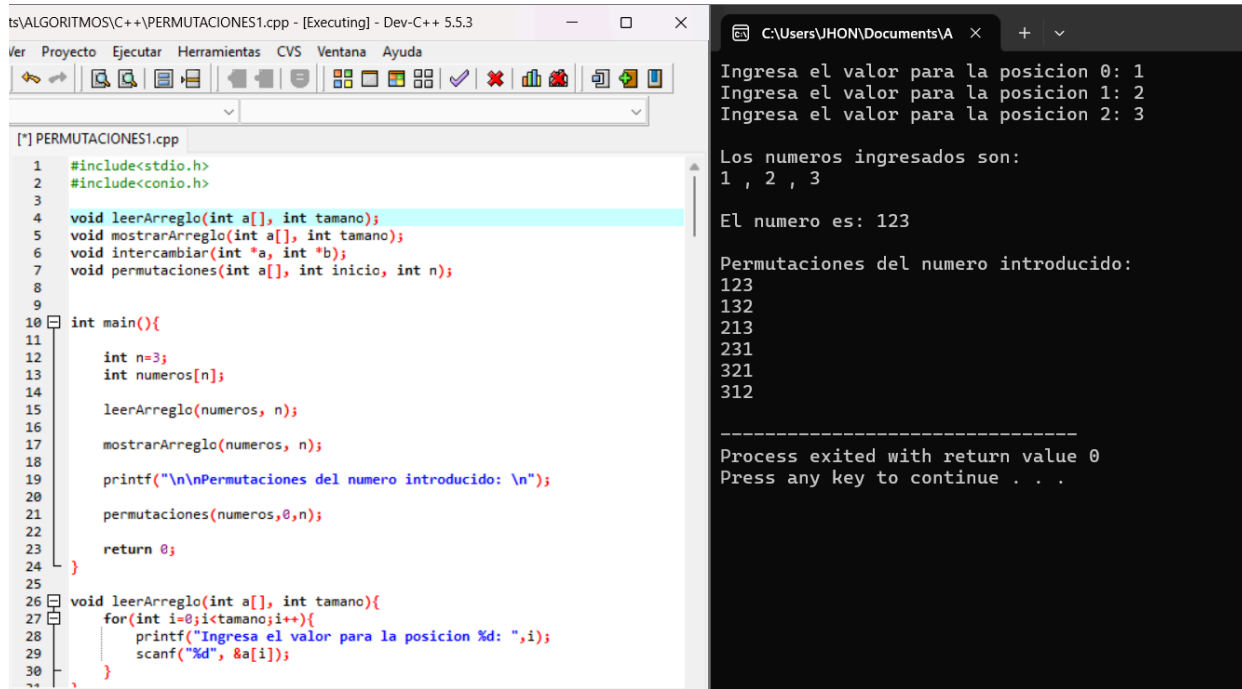


```
1 #include<stdio.h>
2 #include<math.h>
3 #include<conio.h>
4
5 int main(){
6     int a[]={1,4,3,5};
7
8     for(int i=0;i<4;i++){
9         for(int j=0;j<4;j++){
10             if(j==i){
11                 continue;
12             }
13             for(int k=0;k<4;k++){
14                 if(k==i || k==j){
15                     continue;
16                 }
17                 printf("%i%i%i\n",a[i],a[j],a[k]);
18             }
19         }
20     }
21     return 0;
22 }
```

```
143
145
134
135
154
153
413
415
431
435
451
453
314
315
341
345
351
354
514
513
541
543
531
534
```

El resultado es distinto a cuando se usa el backtracking y añadiendo dos funciones más para cuando se desea agregar los números deseados. Para ambos casos se utilizó el mismo conjunto de elementos con el objetivo de comprobar que ambos algoritmos permiten obtener todas las permutaciones.

“Año de la Esperanza y el Fortalecimiento de la Democracia”



```
ts\ALGORITMOS\C++\PERMUTACIONES1.cpp - [Executing] - Dev-C++ 5.5.3
Ver Proyecto Ejecutar Herramientas CVS Ventana Ayuda

[PERMUTACIONES1.cpp]
1  #include<stdio.h>
2  #include<conio.h>
3
4  void leerArreglo(int a[], int tamaño);
5  void mostrarArreglo(int a[], int tamaño);
6  void intercambiar(int *a, int *b);
7  void permutaciones(int a[], int inicio, int n);
8
9
10 int main(){
11
12     int n=3;
13     int numeros[n];
14
15     leerArreglo(numeros, n);
16
17     mostrarArreglo(numeros, n);
18
19     printf("\n\nPermutaciones del numero introducido: \n");
20
21     permutaciones(numeros,0,n);
22
23     return 0;
24 }
25
26 void leerArreglo(int a[], int tamaño){
27     for(int i=0;i<tamaño;i++){
28         printf("Ingresa el valor para la posicion %d: ",i);
29         scanf("%d", &a[i]);
30     }
31 }
```

```
C:\Users\JHON\Documents\A
Ingresa el valor para la posicion 0: 1
Ingresa el valor para la posicion 1: 2
Ingresa el valor para la posicion 2: 3

Los numeros ingresados son:
1 , 2 , 3

El numero es: 123

Permutaciones del numero introducido:
123
132
213
231
321
312

-----
Process exited with return value 0
Press any key to continue . . .
```

COMPLEJIDAD TEMPORAL Y ESPACIAL

El algoritmo tiene una complejidad de $O(n \times n!)$. Esto se debe a que el programa genera todas las posibles permutaciones de los n elementos. La cantidad de permutaciones es $n!$. Además, cada vez que encuentra una permutación, recorre los n elementos para mostrarla. Por ello, el número de operaciones relacionadas con la generación y visualización aumenta aproximadamente como $n \times n!$.

Esto se observa principalmente en la siguiente imagen:

```
void permutaciones(int a[], int inicio, int n){
    if(inicio==n){
        for(int i=0;i<n;i++){
            printf("%d", a[i]);
        }
        printf("\n");
        return;
    }
    for(int i=inicio; i<n; i++){
        intercambiar(&a[inicio], &a[i]);
        permutaciones(a, inicio+1, n);
        intercambiar(&a[inicio], &a[i]);
    }
}
```

La complejidad espacial es $O(n)$. Esto se debe principalmente al arreglo `int numeros[n]` que almacena n elementos. Además, la función `permutaciones()` utiliza recursividad, llegando como máximo n niveles.

COMBINACIONES

DESCRIPCIÓN DEL PROBLEMA

Se tiene un conjunto de n elementos distintos que se agrupan en k elementos sin importar el orden, así que queremos saber todas las combinaciones que podemos obtener:

Ejemplo:

Dado el conjunto (n):

$$\{1, 2, 3\}$$

Agrupación de los elementos (k):

$$k = 2$$

Las posibles combinaciones son:

$$12, 13, 23$$

Por lo tanto, para ($n=3$ y $k=2$):

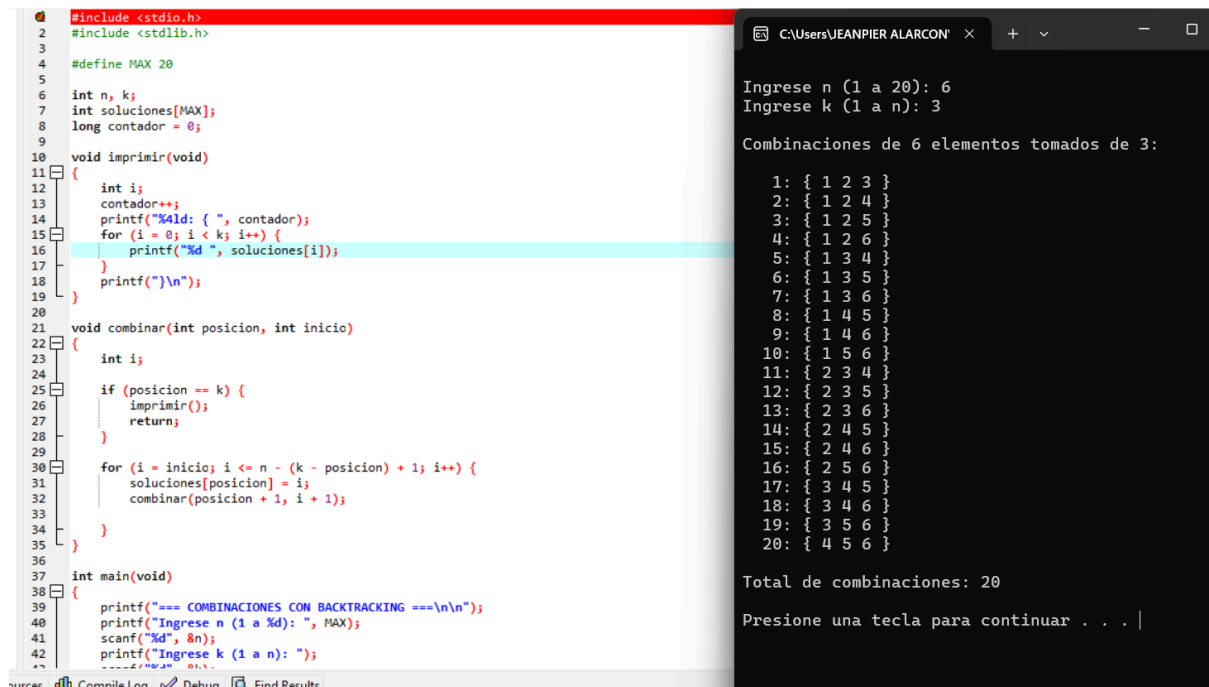
$$3! / 2! (3 - 2)! = 3 \times 2 \times 1 / (1 \times 2)(1) = 3$$

Finalmente, se encontraron 3 posibles combinaciones.

ESTRATEGIA DE BACKTRACKING

Construimos una combinación de elemento a elemento en un arreglo donde se limita la cantidad de elemento por arreglo por la condición k , además empezamos por una variable “inicio” para evitar duplicado. utilizando técnicas de backtracking, hacemos uso de la poda y el retroceso en caso de no ser válida ese camino de solución.

EJEMPLO Y RESULTADO DE EJECUCIÓN



```
#include <stdio.h>
#include <stdlib.h>
#define MAX 20
int n, k;
int soluciones[MAX];
long contador = 0;

void imprimir(void)
{
    int i;
    contador++;
    printf("%4ld: { ", contador);
    for (i = 0; i < k; i++) {
        printf("%d ", soluciones[i]);
    }
    printf("}\n");
}

void combinar(int posicion, int inicio)
{
    int i;
    if (posicion == k) {
        imprimir();
        return;
    }
    for (i = inicio; i <= n - (k - posicion) + 1; i++) {
        soluciones[posicion] = i;
        combinar(posicion + 1, i + 1);
    }
}

int main(void)
{
    printf("=== COMBINACIONES CON BACKTRACKING ===\n\n");
    printf("Ingrese n (1 a 20): ", MAX);
    scanf("%d", &n);
    printf("Ingrese k (1 a n): ");
    scanf("%d", &k);

    combinar(0, 1);

    printf("Total de combinaciones: %ld\n", contador);
    printf("Presione una tecla para continuar . . . |");
}
```

Ingrese n (1 a 20): 6
Ingrese k (1 a n): 3

Combinaciones de 6 elementos tomados de 3:

```
1: { 1 2 3 }
2: { 1 2 4 }
3: { 1 2 5 }
4: { 1 2 6 }
5: { 1 3 4 }
6: { 1 3 5 }
7: { 1 3 6 }
8: { 1 4 5 }
9: { 1 4 6 }
10: { 1 5 6 }
11: { 2 3 4 }
12: { 2 3 5 }
13: { 2 3 6 }
14: { 2 4 5 }
15: { 2 4 6 }
16: { 2 5 6 }
17: { 3 4 5 }
18: { 3 4 6 }
19: { 3 5 6 }
20: { 4 5 6 }
```

Total de combinaciones: 20

Presione una tecla para continuar . . . |

COMPLEJIDAD TEMPORAL Y ESPACIAL

la complejidad temporal es $O(K * C(n, k))$ donde se generan $C(n, k)$ combinaciones y cada una cuesta $O(k)$ al imprimirse. En el peor caso es ($k = n/2$) crece como $O(2^n)$.

La complejidad es $O(k)$ con profundidad máxima de recursión k .

N-REINAS

DESCRIPCIÓN DEL PROBLEMA

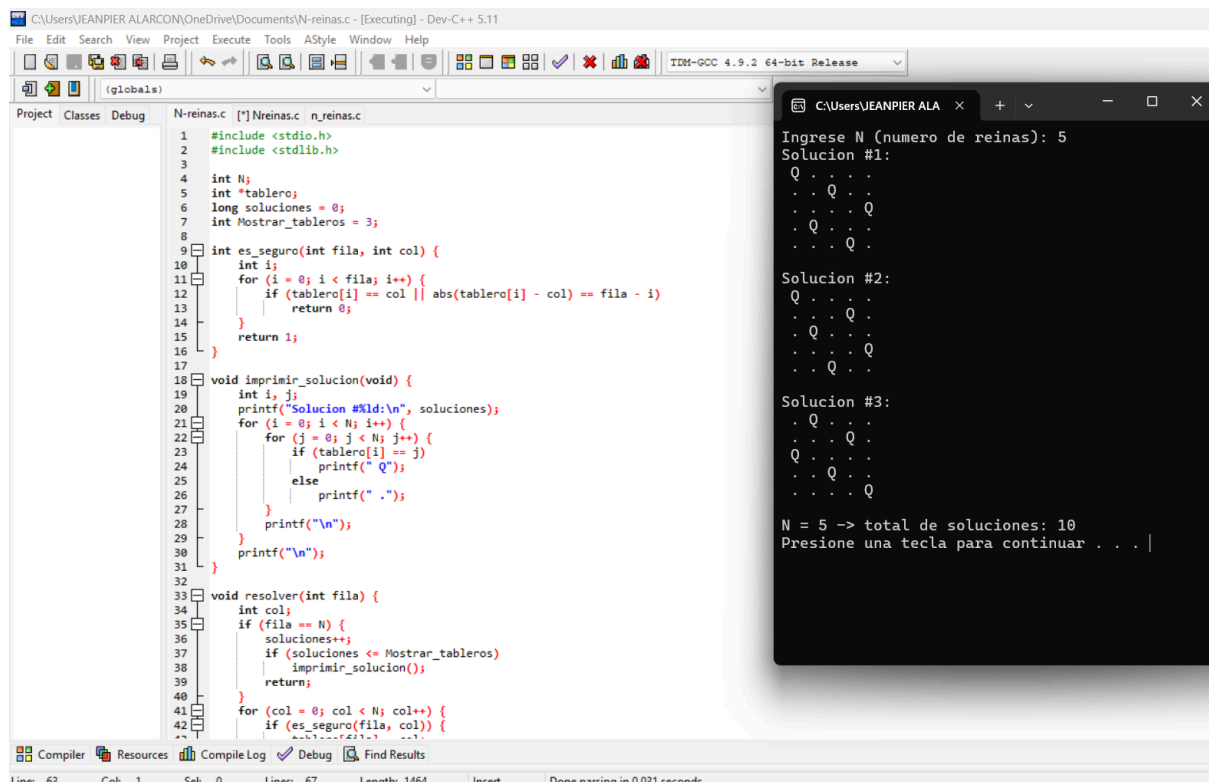
Se desea colocar N reinas de ajedrez en un tablero de tamaño $N \times N$ de tal manera que ninguna reina pueda atacar a otra, es decir, no puede haber dos reinas en una misma fila, columna o diagonal.

ESTRATEGIA DE BACKTRACKING

La técnica de backtracking explora las posibles soluciones verificando fila por fila (f) y de la misma manera columna por columna (c). Asimismo se verifica que no compartan la misma diagonal, aunque para este caso se suelen presentar dos restricciones de la forma $f+c$ y $f-c$. Empezamos por verificar la fila y que en esta se cumpla con las restricciones para así poder pasar recursivamente a la siguiente fila ($f+1$), y en caso de presentar alguna restricción retroceder para seguir probando otra posición o realizar una poda.

EJEMPLO Y RESULTADO DE EJECUCIÓN

Ejecutamos una solución con backtracking con un límite para mostrar 3 soluciones distintas.



```
#include <stdio.h>
#include <stdlib.h>

int N;
int *tablero;
long soluciones = 0;
int Mostrar_tableros = 3;

int es_seguro(int fila, int col) {
    int i;
    for (i = 0; i < fila; i++) {
        if (tablero[i] == col || abs(tablero[i] - col) == fila - i)
            return 0;
    }
    return 1;
}

void imprimir_solucion(void) {
    int i, j;
    printf("Solucion #%ld:\n", soluciones);
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (tablero[i] == j)
                printf("Q");
            else
                printf(".");
        }
        printf("\n");
    }
    printf("\n");
}

void resolver(int fila) {
    int col;
    if (fila == N) {
        soluciones++;
        if (soluciones <= Mostrar_tableros)
            imprimir_solucion();
        return;
    }
    for (col = 0; col < N; col++) {
        if (es_seguro(fila, col)) {
            tablero[fila] = col;
            resolver(fila + 1);
        }
    }
}

int main() {
    printf("Ingrese N (numero de reinas): ");
    scanf("%d", &N);
    tablero = (int *) malloc(N * sizeof(int));
    resolver(0);
    printf("N = %d -> total de soluciones: %ld\n", N, soluciones);
    printf("Presione una tecla para continuar . . . |");
    getch();
    return 0;
}
```

COMPLEJIDAD TEMPORAL Y ESPACIAL

La complejidad temporal es $O(N^2 \times N!)$ debido a las n columnas posibles y las N filas disponibles.

La complejidad espacial es $O(N)$ debido a que la recursión llega como máximo a N niveles.

SUDOKU

DESCRIPCIÓN DEL PROBLEMA

El sudoku consiste en rellenar una cuadrícula de 9x9 casillas con números del 1 al 9, cumpliendo determinadas restricciones. Cada fila debe contener los números del 1 al 9 sin repetirlos, al igual que cada columna. Además, cada subcuadrícula de 3x3 debe contener los números del 1 al 9 sin repetición.

El juego comienza con algunas casillas previamente llenas que sirven como guía y que no pueden ser modificadas. El objetivo es completar todas las casillas vacías respetando las restricciones mencionadas.

9		1	3					
				7	4			
		3	1	5				
2								
			9			7		8
3	5			8		4		2
4								3
1					7	5		
8		5						6

ESTRATEGIA DE BACKTRACKING

Para resolver el sudoku se implementaron 2 algoritmos, uno sin usar el Backtracking y otro usándolo.

En el algoritmo sin Backtracking se recorren las casillas vacías y se coloca el primer número que cumple las restricciones de fila, columna y subcuadrícula. Pero, si una decisión tomada anteriormente conduce posteriormente a una situación en la que no existe ningún número válido, entonces el algoritmo no puede regresar y modificar dicha decisión.

En cambio, el algoritmo con Backtracking utiliza una estrategia de búsqueda recursiva. Se coloca un número válido en una casilla vacía y se intenta resolver el resto del sudoku. Si posteriormente se llega a una situación sin solución se deshace la última decisión y se prueba con otro número.

EJEMPLO Y RESULTADO DE EJECUCIÓN

Se ejecutaron ambos algoritmos utilizando el mismo sudoku inicial para comparar su comportamiento. En la ejecución se puede observar la diferencia entre ambos métodos mediante las salidas obtenidas por cada programa.

“Año de la Esperanza y el Fortalecimiento de la Democracia”

```
1 #include <stdio.h>
2 #define N 9
3
4 void imprimirSudoku(int sudoku[N][N]);
5 int esValido(int sudoku[N][N], int fila, int columna, int numero);
6 void resolverSinBacktracking(int sudoku[N][N]);
7
8 int main() {
9     int sudoku[N][N] = {
10         {5, 3, 0, 0, 7, 0, 0, 0, 0},
11         {6, 0, 0, 1, 9, 5, 0, 0, 0},
12         {0, 9, 8, 0, 0, 0, 0, 6, 0},
13         {8, 0, 0, 0, 6, 0, 0, 0, 3},
14         {4, 0, 0, 8, 0, 3, 0, 0, 1},
15         {7, 0, 0, 0, 2, 0, 0, 0, 6},
16         {0, 6, 0, 0, 0, 0, 2, 8, 0},
17         {0, 0, 0, 4, 1, 9, 0, 0, 5},
18         {0, 0, 0, 0, 0, 0, 0, 7, 9}
19     };
20
21     printf("SUDOKU SIN BACKTRACKING\n\n");
22
23     printf("Sudoku inicial:\n");
24     imprimirSudoku(sudoku);
25
26     resolverSinBacktracking(sudoku);
27
28     printf("\nSudoku despues del metodo:\n");
29     imprimirSudoku(sudoku);
30
31     return 0;
32 }
```

Sudoku inicial:

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	2	8	0
0	0	0	4	1	9	0	0	5
0	0	0	0	0	0	0	7	9

Sudoku despues del metodo:

5	3	1	2	7	4	8	9	0
6	2	4	1	9	5	3	0	7
0	9	8	3	0	0	1	6	2
8	1	2	5	6	7	4	0	3
4	5	6	8	0	3	7	2	1
7	0	3	9	2	1	5	0	6
1	6	5	7	3	0	2	8	4
2	7	0	4	1	9	6	3	5
3	4	0	6	8	2	0	7	9

Process exited with return value 0
Press any key to continue . . .

```
1 #include <stdio.h>
2 #define N 9
3
4 void imprimirSudoku(int sudoku[N][N]);
5 int esValido(int sudoku[N][N], int fila, int columna, int numero);
6 int resolverSudoku(int sudoku[N][N]);
7
8 int main() {
9     int sudoku[N][N] = {
10         {5, 3, 0, 0, 7, 0, 0, 0, 0},
11         {6, 0, 0, 1, 9, 5, 0, 0, 0},
12         {0, 9, 8, 0, 0, 0, 0, 6, 0},
13         {8, 0, 0, 0, 6, 0, 0, 0, 3},
14         {4, 0, 0, 8, 0, 3, 0, 0, 1},
15         {7, 0, 0, 0, 2, 0, 0, 0, 6},
16         {0, 6, 0, 0, 0, 0, 2, 8, 0},
17         {0, 0, 0, 4, 1, 9, 0, 0, 5},
18         {0, 0, 0, 0, 0, 0, 0, 7, 9}
19     };
20
21     printf("SUDOKU SIN BACKTRACKING\n\n");
22
23     printf("Sudoku inicial:\n");
24     imprimirSudoku(sudoku);
25
26     if (resolverSudoku(sudoku)) {
27         printf("\nSudoku resuelto:\n");
28         imprimirSudoku(sudoku);
29     }
30
31     return 0;
32 }
```

Sudoku inicial:

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	2	8	0
0	0	0	4	1	9	0	0	5
0	0	0	0	0	0	0	7	9

Sudoku resuelto:

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Process exited with return value 0
Press any key to continue . . .

COMPLEJIDAD TEMPORAL Y ESPACIAL

La complejidad temporal del algoritmo es $O(9^k)$, donde k es el número de casillas vacías. Esto se debe a que el programa prueba hasta 9 números en cada casilla y realiza una llamada recursiva para continuar buscando una solución. Operando en el peor caso se pueden explorar $9 \times 9 \times 9 \times \dots \times 9 = 9^{81}$ posibilidades. La función `esValido()` permite descartar

“Año de la Esperanza y el Fortalecimiento de la Democracia”

opciones que no cumplen las reglas del sudoku, reduciendo la cantidad de caminos explorados.

```
89 int resolverSudoku(int sudoku[N][N]) {
90     int fila, columna;
91     int numero;
92
93     // Buscar una casilla vacia
94     for (fila = 0; fila < N; fila++) {
95         for (columna = 0; columna < N; columna++) {
96
97             if (sudoku[fila][columna] == 0) {
98
99                 // Probar numeros del 1 al 9
100                 for (numero = 1; numero <= 9; numero++) {
101
102                     if (esValido(sudoku, fila, columna, numero)) {
103                         // Colocar numero
104                         sudoku[fila][columna] = numero;
105
106                         // Intentar resolver el resto
107                         if (resolverSudoku(sudoku)) {
108                             return 1;
109                         }
110                         // Si no funciona, regresar
111                         sudoku[fila][columna] = 0;
112                     }
113                 }
114                 return 0;
115             }
116         }
117     }
118     // No quedan casillas vacias
119     return 1;
120 }
```

La complejidad espacial es $O(k)$, debido a las llamadas recursivas de la función `resolverSudoku()`. Cada llamada ocupa memoria, y en el peor caso, puede haber una llamada por cada casilla vacía. El tablero 9x9 ocupa un espacio constante.

COMPARACIÓN DE LOS ALGORITMOS IMPLEMENTADOS

COMPARACIÓN DE COMPLEJIDAD

Con el uso del backtracking la complejidad temporal disminuye en gran medida brindándonos una respuesta más rápida y eficiente. Por otro lado, con respecto a la complejidad espacial se sigue manteniendo igual debido a que estos se ven condicionados por el mismo ingreso.

COMPARACIÓN DE CARACTERÍSTICAS

Ambos métodos, sin backtracking y con backtracking, llegan a obtener los mismos resultados aunque la principal diferencia se observa en el tiempo que tardan en mostrarnos las soluciones, debido a que la técnica de backtracking hace uso de la poda y el retroceso para evitar seguir caminos que no conllevan a soluciones válidas, por tal manera el tiempo de ejecución se nota e influyen en la escalabilidad de cada algoritmo.

CONCLUSIONES

El backtracking es uno de los métodos más utilizados y favorables para la creación de algoritmos que requieren explorar diferentes posibilidades para encontrar una solución. Su principal característica es retroceder una vez que sepas que hay algo malo en el camino elegido, por lo tanto no seguir con una alternativa que no vaya a proporcionar una solución. Gracias al uso de esta técnica en el diseño de permutaciones y en la resolución del sudoku podemos ver como nos permite explorar diversas posibilidades y descartar las que no satisfacen.

Además, las herramientas como la poda es vital al momento de ser eficientes y tener una mayor escalabilidad, es notorio la influencia que tiene en la búsqueda de combinaciones y el problema de las N-reinas ya que nos evita seguir caminos de soluciones erradas que tomarían mucho tiempo de ejecución.